Bug Hunt v2-3: Bank Ledger

What the program does

This problem consists of **two modules** plus a driver:

- `account.erl` — a single-account ADT. An account is represented as {`account, Name, Balance`}. Exports: `new/2`, `name/1`, `balance/1`, `deposit/2`, `withdraw/2`.
- `ledger.erl` — a multi-account ledger, implemented as a list of accounts. Exports: `new/0`, `add_account/2`, `find/2`, `transfer/4`, `apply_all/2`, `balance/2`, `total_assets/1`, `list_balances/1`.
- `main.erl` — builds a 3-account ledger (alice, bob, carol), applies a sequence of 4 transactions, and prints the state before and after.

Intended account behavior:

- `new(Name, Balance)` creates an account with the given initial balance.
- `balance/1` returns the account's current balance.
- `deposit(Account, Amount)` returns a new account with `Amount` added to the balance. `Amount` must be non-negative.
- `withdraw(Account, Amount)` returns {`ok, NewAccount`} if the account has at least `Amount` available, otherwise returns {`error, insufficient_funds`} and the account is unchanged.

Intended ledger behavior:

- `transfer(Ledger, FromName, ToName, Amount)` debits `Amount` from `FromName`'s account and credits the same `Amount` to `ToName`'s account. Returns {`ok, NewLedger`} on success; {`error, Reason`} on any failure (account not found, insufficient funds). On error, the ledger is unchanged.
- `apply_all/2` runs a list of transactions in order; failed transactions are skipped (the next transaction continues from the pre-failure state).
- `total_assets/1` returns the sum of all balances in the ledger.
- `list_balances/1` returns `[{Name, Balance}]` for every account.

Two important invariants:

1. **Conservation of money.** A successful transfer never changes the total assets. Failed transfers also never change the total.
2. **No negative balances and no overdrafts.** Every `balance/1` value is ≥ 0 at all times.

The catch

This program contains **exactly 4 bugs** that prevent it from producing the expected output. The bugs are all *semantic* — the program compiles without errors and runs to completion without crashing.

This problem is harder than the round-1 bug hunts in four specific ways:

1. **At least one bug's cause lives in a different file from where its symptom appears.** `ledger:transfer` *looks* like it does the right thing — and yet, depending on which other bugs are present, it can either do nothing at all or do exactly the opposite of what it should.
2. **The bugs form a hiding chain.** As-shipped, one loud bug hides the symptoms of others. Fixing it does not produce correct output — it *reveals* the next layer of wrong output. You may need to fix several bugs before you can clearly attribute any symptom to a specific cause.
3. **One of the bugs is conservation-preserving.** Any sanity check based on "did the total stay the same?" will not detect it. You have to compare individual balances against your prediction.
4. **One bug is dormant in the as-shipped state and only fires after another bug is fixed.** If, after fixing what looks like the most obvious bug, a new symptom appears that wasn't there before, do not assume you broke something — assume there are more bugs to find.

You may not run the program until you have submitted your answer. Trace by reading.

Expected output

```
Initial state:
  Balances:
    carol: $50
    bob: $200
    alice: $100
  Total assets: $350

Applying transactions:
  transfer alice -> bob: $30
  transfer bob -> carol: $50
  transfer alice -> carol: $200
  transfer carol -> alice: $25

Final state:
  Balances:
    carol: $75
    bob: $180
    alice: $95
  Total assets: $350

Direct balance queries:
  alice: $95
  bob:   $180
  carol: $75
```

Notes on the expected output:

- Balances are printed in the order accounts appear in the ledger's internal list (most recently added first, so carol appears before bob, which appears before alice).
- The third transaction (`alice -> carol $200`) should fail with insufficient funds: by the time T3 runs, alice's balance is $70 (started at $100, sent $30 to bob in T1). The transfer is skipped and balances do not change.
- The expected output shows total assets unchanged ($350) at both the start and the end — this is the conservation-of-money invariant.

Your write-up

For each bug, record:

1. **Location** — file name and line number(s).
2. **Diagnosis** — in 1–2 sentences, what is wrong.
3. **Fix** — the corrected line(s).
4. **Symptom** — which line(s) of the expected output differ from what the buggy program actually prints, and why this specific bug causes that specific symptom. If a bug has no *direct* visible symptom (because another bug masks it), explain how you knew it was there anyway.